

EL403 – Assignment 2

Karthik Maddimsetti (202304006) and Het Patel (202304033), *Dhirubhai Ambani University, Gandhinagar.*

About the document — In this document, we have presented our understanding, analysis and conclusions on the given exercise, with all the discussions relating to Array and Wallace Tree Multipliers. The simulation results are based on the transition dependent delays taken from Cadence designs of basic gates made during the last semester.

COMMON METHODOLOGY

All circuits described in this report were modeled in structural Verilog using basic logic gates whose delays were obtained from full-custom designs created and simulated in Cadence Virtuoso. The gate delays were *transition-dependent*, with separate rise and fall values. These delays were incorporated into the Verilog gate models using **specify** blocks, to ensure that the timing behavior in simulation closely matched the transistor-level implementation. Functional and timing simulations were performed in Xilinx Vivado, to observe delay effects.

The values of the delays obtained from the Cadence simulations are shown in Appendix[3].

To find the worst-case delay combination and delays of different inputs of an implementation, we have simulated all the possible input combinations using the code mentioned in Appendix[1]. Then used Microsoft Excel to generate the histograms and summary used to conclude the advantages and limitations in III.B.

I. ARRAY MULTIPLIER

Array multiplier works by generating partial products for each bit of the multiplier and then adding them together in a structured “array” format using ripple carry adders to produce the final product.

A. Inputs:

$A[m-1:0]$ – an m-bit multiplicand

B. Output:

$P[m+n-1:0]$ – (m+n) bit product

C. Structure:

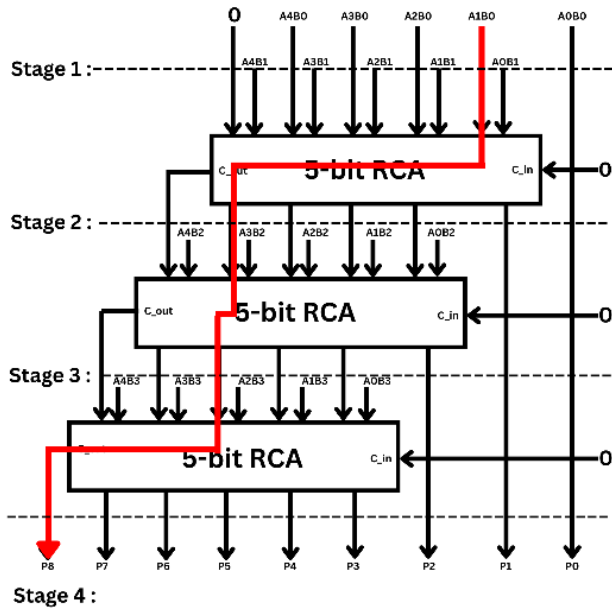
The whole multiplier structure can be broadly divided into two types of stages:

- **Partial-Product Generation Stage:** for an $m \times n$ multiplier, a total of $m \times n$ partial products are generated using $m \times n$ AND gates.
- **Ripple Carry Adder Stages:** for an $m \times n$ multiplier, a total of (n-1) ripple carry adders of m-bit length are used.

$$\text{Length of RCA} = m$$

$$\text{Number of Stages} = (n - 1)$$

Structure of the 5×4 multiplier is shown in the figure below.

Figure 1.1 : 5×4 Array multiplier**D. Critical Path:**

The total delay of the $m \times n$ multiplier is given by the delay of the last product bit.

$$T_{mult} \approx T_{AND} + (M + N - 2)T_{carry} + (N - 2)T_{sum} \quad (1.1)$$

where,

T_{AND} is Delay of the AND gate

T_{carry} is worse case carry-out delay of FA

T_{sum} is worse case sum-out delay of FA

T_{mult} is the total delay of the multiplier

E. 5×4 Array multiplier:

As shown Fig. 1.1, in a 5×4 Array multiplier there are 3 5-bit RCAs and 20 AND gates to generate partial products.

Critical path:

The critical path of the multiplier is marked in Fig. 1.1.

Simulation Results:

We simulated all the input combinations of A and B for the circuit, and found that the maximum delay for the combination $A = 5'b11111$ and $B = 4'b1001$.

$$T_{mult} = T_{(A,B) \rightarrow P} = 18.262 \text{ ns}$$

Verifying stage-wise delays,

$$T_1 = 1.185 \text{ ns}$$

$$T_{1 \rightarrow 2} = 8.453 \text{ ns}$$

$$T_{2 \rightarrow 3} = 1.720 \text{ ns}$$

$$T_{3 \rightarrow 4} = 6.904 \text{ ns}$$

$$T_{mult} = T_1 + T_{1 \rightarrow 2} + T_{2 \rightarrow 3} + T_{3 \rightarrow 4} = 18.262 \text{ ns}$$

$$F_{max} = \frac{1}{T_{mult}} 4.758 \text{ MHz}$$

F. 4×5 Array multiplier:

For a 4×5 Array multiplier total of 4 4-bit RCAs are used, and 20 partial products are generated using AND gates.

$$\text{Length of each RCA} = m = 4$$

$$\text{Number of stages} : (n-1) = 4$$

Simulation Results:

In the same way as stated for 5×4 multiplier, we found the worst case combination for 4×5 multiplier to be $A = 4'b1011$ and $B = 5'b11011$.

$$T_{mult} = T_{(A,B) \rightarrow P} = 19.156 \text{ ns}$$

Verifying stage-wise delays,

$$T_1 = 1.185 \text{ ns}$$

$$T_{1 \rightarrow 2} = 9.207 \text{ ns}$$

$$T_{2 \rightarrow 3} = 0.172 \text{ ns}$$

$$T_{3 \rightarrow 4} = 4.55 \text{ ns}$$

$$T_{4 \rightarrow 5} = 4.042 \text{ ns}$$

$$T_{mult} = T_1 + T_{1 \rightarrow 2} + T_{2 \rightarrow 3} + T_{3 \rightarrow 4} + T_{4 \rightarrow 5} = 19.156 \text{ ns}$$

$$F_{max} = \frac{1}{T_{mult}} 2.203 \text{ MHz}$$

As we can see, 4×5 multiplier has lesser delay compared to 5×4 multiplier.

Conclusion:

If $m > n$ then, the $m \times n$ multiplier will give faster output compared to $n \times m$.

Because if the multiplier has a greater number of bits than the multiplicand, then the Array multiplier will have more number of stages than its converse, which leads to the final output being delayed further. As we can see in equation 1.1, if the multiplier has more bits, then the coefficient of T_{sum} i.e., $(N - 2)$ will increase, which leads to a greater delay.

II. WALLACE TREE MULTIPLIER

In a $m \times n$ Wallace Tree Multiplier, we reduce the maximum number of possible bits in a column using FAs (3:2 compressors) and HAs (2:2 compressors) until the maximum number of bits in any column is 2. Later these two rows are fed into a fast adder to merged them into a sum which is the product of the performed multiplication.

A. Inputs:

- $A[m-1:0]$ – an m-bit multiplicand and
- $B[n-1:0]$ – an n-bit multiplier

B. Output:

- $P[m+n-1:0]$ – $(m+n)$ bit product

C. Structure:

The whole multiplier structure can be broadly divided into two types of stages:

- Column Reduction Stage (CRS): The stages from the first stage to the last stage in which the bits in each column are reduced successively using FAs and HAs.
- Row Reduction Stage (RRS): The last stage in which the remaining rows are merged using a fast adder or a parallel prefix adder.

$$T_{mult} = T_{PP} + T_{CRS} + T_{RRS}$$

$$T_{CRS} = t_0 + t_1 + t_2 + \dots + t_{s-1}$$

$$t_i = \max(t_{HA}, t_{FA}), 0 \leq i \leq s - 1$$

where,

T_{PP} is the delay of the partial product generation,

T_{mult} is the Total delay of the Multiplier,

T_{RRS} is the delay of the Row reduction stage, and

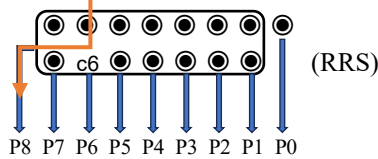
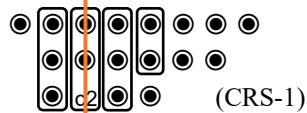
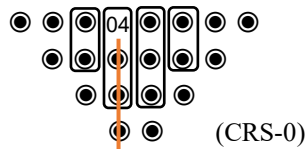
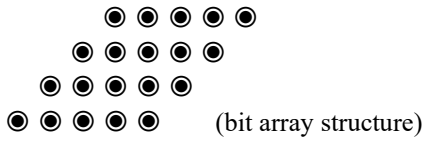
T_{CRS} is the Total delay of all the Column reduction stages

t_{HA}, t_{FA} are the worse case delays of FA and HA respectively

$t_1, t_2, t_3, \dots, t_s$ are the delays of the individual column reduction stages and

s is the number of Column reduction stages.

Before getting into generalizing the structure to an $m \times n$ multiplication let us consider a simpler example, taking $m = 5$ and $n = 4$.



- RRS is the Row Reduction Stage
- CRS is a Column Reduction Stage

In the above diagram, the marked path is one of the critical paths possible. Also, we can understand the following about the 5×4 Wallace Multiplier from the figure,

(i) $s = 2$,

(ii) $T_{PP} = T_{04}$, where T_{04} is the delay of the partial product, $A[0]$ AND $B[4]$

(iii) $t_1 = \llbracket (t) \rrbracket_{c2} - T_{PP}$ and $t_2 = \llbracket (t) \rrbracket_{c6} - t_{c2}$

(iv) $T_{RRS} = \max(T_{sum}, T_{cout}) - t_{c6}$ i.e., worst case propagation delay of the 7-bit fast adder employed

(v) we'll need 20 AND gates, 3 HAs, 5FAs and 7-bit fast adder to implement this circuit.

Simulation Results:

We have first plugged in a 7-bit RCA as the fast adder in RRS. The following are the results of the simulation for the worst-case delay inputs, $A = 5'b11111$ and $B = 4'b1000$.

Worst-case delay:

$$T_{(A,B) \rightarrow P} = T_{mult} = 14.528 \text{ ns}$$

Checking the results stage-wise,

$$T_{PP} = 1.049 \text{ ns}$$

$$t_{c2} = 2.782 \text{ ns}$$

$$t_{c6} = 4.703 \text{ ns}$$

$$T_{cout} = 14.528 \text{ ns}$$

$$t_1 = 2.782 \text{ ns} - 1.049 \text{ ns} = 1.733 \text{ ns}$$

$$t_2 = 4.703 \text{ ns} - 2.782 \text{ ns} = 1.921 \text{ ns}$$

$$T_{RRS} = 14.528 \text{ ns} - 4.703 \text{ ns} = 9.825 \text{ ns}$$

Therefore,

$$T_{mult} = 1.049 \text{ ns} + 1.733 \text{ ns} + 1.921 \text{ ns} + 9.825 \text{ ns} = 14.528 \text{ ns}$$

confirming that the taken path is the critical path of the circuit.

Hence,

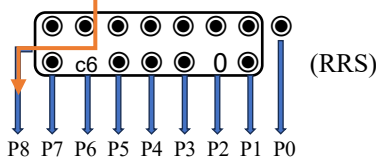
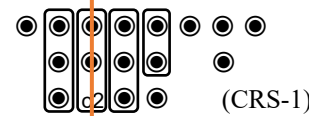
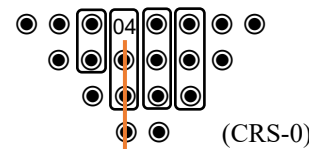
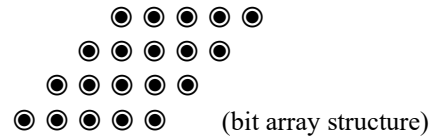
The maximum possible clock frequency,

$$F_{max} = \frac{1}{T_{mult}} = \frac{1}{14.528} \text{ ns} = 68.832 \text{ MHz}.$$

If sum and carry stages are not balanced:

We have intentionally replaced one of the HAs in CRS-0 with a FA to create an imbalance between the sum and carry stages.

The tree structure looks as follows:



Simulation Results:

We have first plugged in a 7-bit RCA as the fast adder in RRS. The following are the results of the simulation for the worst case delay inputs, $A = 5'b11111$ and $B = 4'b1000$,

Worst case delay:

$$T_{(A,B) \rightarrow P} = T_{mult} = 14.682 \text{ ns}$$

Checking the results stage-wise,

$$T_{PP} = 1.185 \text{ ns}$$

$$t_{c2} = 2.785 \text{ ns}$$

$$t_{c6} = 3.955 \text{ ns}$$

$$T_{cout} = 14.682 \text{ ns}$$

$$t_1 = 2.785 \text{ ns} - 1.185 \text{ ns} = 1.610 \text{ ns}$$

$$t_2 = 3.955 \text{ ns} - 2.785 \text{ ns} = 1.160 \text{ ns}$$

$$T_{RRS} = 14.682 \text{ ns} - 3.955 \text{ ns} = 10.727 \text{ ns}$$

Therefore,

$T_{mult} = 1.185 \text{ ns} + 1.610 \text{ ns} + 1.160 \text{ ns} + 10.727 \text{ ns} = 14.682 \text{ ns}$ confirming that the taken path is the critical path of the circuit.

Hence,

The maximum possible clock frequency,

$$F_{max} = \frac{1}{T_{mult}} = \frac{1}{14.528} \text{ ns} = 68.111 \text{ MHz.}$$

Similarly, when plugging in a 7-bit CLA (4-bit CLA and 3-bit CLA in sequential) into a Balanced Wallace tree, we got the following simulation results for the worst combination, $A = 5'b01011$ and $B = 4'b1100$.

$$T_{mult} = 12.741 \text{ ns}$$

$$F_{max} = 78.486 \text{ MHz}$$

Stage-wise delays,

$$T_{PP} = 1.184 \text{ ns}$$

$$t_{c1} = 4.063 \text{ ns}$$

$$t_{c5} = 5.984 \text{ ns}$$

$$T_{P6} = 12.741 \text{ ns}$$

$$t_1 = 2.879 \text{ ns}$$

$$t_2 = 1.921 \text{ ns}$$

$$T_{RRS} = 6.757 \text{ ns}$$

Note that, CRS are slower than RCA case but RRS is faster. because we have found the worst case only, in this case, Slower CRS is leading to worst inputs to RRS. In the other cases it may the other way around.

Likewise, for an unbalanced Wallace tree,

$$T_{mult} = 12.516 \text{ ns}$$

$$F_{max} = 79.897 \text{ MHz}$$

Stage-wise delays,

$$T_{PP} = 1.185 \text{ ns}$$

$$t_1 = 2.878 \text{ ns}$$

$$t_2 = 1.921 \text{ ns}$$

$$T_{RRS} = 6.532 \text{ ns}$$

Observations:

- For RCA cases, the Balanced Wallace tree has a lower a delay and higher F_{max} than the unbalanced version because the higher delay of the extra FA in CRS-0. This delay in carry is getting propagated through the RCA making the merging stage slower.
- In Unbalanced Wallace tree – CLA, CRS are slower than RCA case but RRS is faster, because in the worst case, slower CRS are leading to worst inputs to RRS due the data-dependent delays of the gates used. But, still the multiplier is fast because of the speed of the much lower worst case of the CLA (Faster in RRS stage).
- Comparison between balanced and unbalanced Wallace – CLA multipliers:*
From the delay numbers, it can be understood that the unbalanced has lower worst-case delay than the balanced version.
Reason : Because the unbalanced stage of RRS filled with 0, making that input to the CLA always available so the cout of the 4-bit CLA is computed faster since it used propagate and generate logic. So the computation in 3-bit CLA will start before the 4-bit CLA produced its sum making the computation of the final product faster than the balanced version.
- But the same won't happen in case of the RCA because of the ripple effect. An early input is available only to one of the FAs but the other FAs will still have to wait for the cout from the previous bits.
- Few corner cases where unbalanced product is computed faster than the balanced case: (For RCA)

$$A = 5'b11111, B = 4'b0001$$

$$A = 5'b11111, B = 4'b1000$$

$$A = 5'b01101, B = 4'b0101$$

$$A = 5'b10000, B = 4'b1000$$

Generalization to m x n Wallace Tree Multiplier:

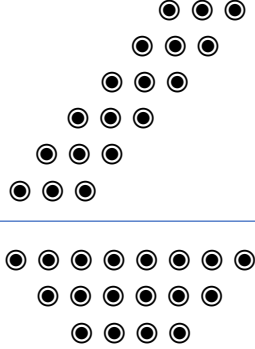
For specific values of m and n, any m x n multiplier folds back to same tree structure as n x m in such way that the CRS-0 has number of rows equal to the minimum of m and n, i.e.,

$$r_0 = \min(m, n)$$

For example, $m = 6$ and $n = 3$,
 6×3



3×6



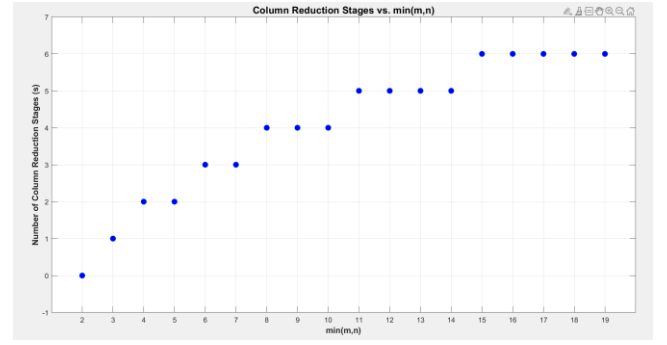
But as the $\max(m,n)$ increases, the number of FAs and(or) HAs increases used in the particular stage increases but this will not significantly affect the worst case delay because all the FAs and HAs in the same stage in parallel. This gives a significant advantage the Wallace Tree structure.

Sublinear variation of s in logarithmic order with $\min(m,n)$:
 We know that¹,

$$r_{i+1} = 2 * \text{floor} \left(\frac{r_j}{3} \right) + r_j \bmod(3) \quad (2.1)$$

where $r_0 = \min(m,n)$ and
 r_i is the number of stages in i^{th} CRS.

Also, s is the number of Column reduction stages, CRS till such that the last stage has 2 rows i.e., $r_{s-1} = 2$. (2.2)
 (2.1) and (2.2) can be used to find the number of value of s for any $m \times n$ Wallace tree multiplier. Therefore, the following plot shows how s sublinearly increases with increase in r_0 .



From the above plot, we can understand that the s will remain constant over a larger range of r_0 .

Since the delay and dependencies increase with number of stages, we can say that, a higher bit (say, 64-bit) Wallace tree multiplier will still have lesser number stages and hence, a better performance than a straight forward Array Multiplier.

III. CONCLUSIONS

A. Advantage of Wallace Tree Multiplier over Array Multiplier:

Plugging in Fast Adders like Kogge Stone, Brent-Kung or CLA into Wallace will be more beneficial as compared to Array, because inputs to the fast adder will be computed in parallel and will available in lesser unwanted transitions. This can significantly reduce the delay of the multiplier due to fast addition stage.

For longer bit word, Wallace tree multipliers have significant performance advantage over array structures. (see *Sublinear variation of s in II*).

More number of unwanted transitions in Array Multiplier than Wallace tree multiplier which may cause to the increased power dissipation.

B. Applications of Multipliers:

Though the functionality is same and satisfied the different implementations of the multipliers in I and II, since the timing is data dependent, *statistically analyzing the delays* of all the possible input combination will give a better idea of strengths of the implementation. That will help us taking a decision on the application of a particular structure.

As mentioned in Common Methodology, we have used the code in Appendix [1] to simulate all the possible input combinations and analyzed the data of the delays. The summary of the data is shown in the Table-1.

¹ This expression is taken from , Chapter-11, Handbook of Digital CMOS Technology, Circuits, and Systems, Karim Abbas.

Table - 1:

	Mean Delay (ns)	Median Delay (ns)	Mean-Median (ns)	Standard Deviation	Maximum Delay (ns)	Maximum Frequency (MHz)
Balanced Wallace - RCA - 5 x 4	9.67088	9.419	0.251875	1.47348754	14.528	68.83259912
Unbalanced Wallace - RCA - 5 x 4	9.66089	9.488	0.172890625	1.58158531	14.682	68.11061163
Balanced Wallace-CLA - 5 x 4	9.38179	9.3025	0.079292969	0.80886673	12.741	78.48677498
Unbalanced Wallace-CLA - 5 x 4	9.46104	9.255	0.206044922	0.88185489	12.516	79.8977309
Array - 5 x 4	12.4672	12.0225	0.444730469	2.17558821	18.262	54.75851495
Array - 4 x 5	13.2508	13.1	0.150818359	2.51704576	19.156	52.20296513

See Appendix for Histograms plotted for Number of Input Combinations v/s Delay intervals, (bin size = 0.5 ns).

- If *mean and median difference* of an implementation is greater than the other, there are more number of slow combinations in the former.
- Lesser *mean delay* means the circuit gives faster output for most of the input combinations but the it may have a higher maximum delay which means the critical path is slower.
- *Standard deviation* shows how much delays for various inputs varies. This can be related to timing predictability.

The table clearly shows the *trade-off between average-case delay and worst-case delay*. For eg, Balanced Wallace CLA gives the lowest mean delay but not the lowest worst-case delay.

Choice of the multiplier will depend on the specific application,

- Wallace does faster computation but has a huge hardware requirement because of the Fast adder stage.
- Array has regular structure and less hardware requirement but computation is slower because of the dependent stages.

APPENDIX

[1] Code used for finding the worst case delay combination for the balanced and unbalanced versions of the 5 x 4 Wallace tree Multiplier

```

`timescale 1ns/1ps

module tb_auto_5x4;

    reg [4:0] A;
    reg [3:0] B;
    wire [8:0] P;

    wallace_5x4 dut (
        .A(A),
        .B(B),
        .P(P)
    );

    // use Wallace_5x4_unb for the unbalanced
    version of the multiplier

    integer i, j;
    reg [8:0] P_expected;

    time start_t, end_t;
    time delay;
    time max_delay = 0;
    reg [4:0] worst_A;
    reg [3:0] worst_B;

    initial begin

        // Initialize
        A = 0; B = 0;
        #10;

        // loop to apply all combinations of
    inputs
        for (i = 0; i < 32; i = i + 1) begin
            for (j = 0; j < 16; j = j + 1) begin
                P_expected = i * j;

                // initialize inputs to x before applying new
                vector so that we get accurate transitions
                since we have considered data-dependent
                transitions

                A = 5'bx;
                B = 4'bx;
                #50; // delay for x value

                // Mark start time
                start_t = $realtime;
                A = i;
                B = j;

                // Wait until output matches
    expected
                wait (P === P_expected);

                // Measure propagation delay
                end_t = $realtime;
                delay = end_t - start_t;

                if (delay > max_delay) begin
                    max_delay = delay;
                    worst_A = A;
                    worst_B = B;
                end
                $display("A=%0d, B=%0d,
                Delay=%0.3f ns", A, B, delay);
                #10;
            end
        end

    end

end

```

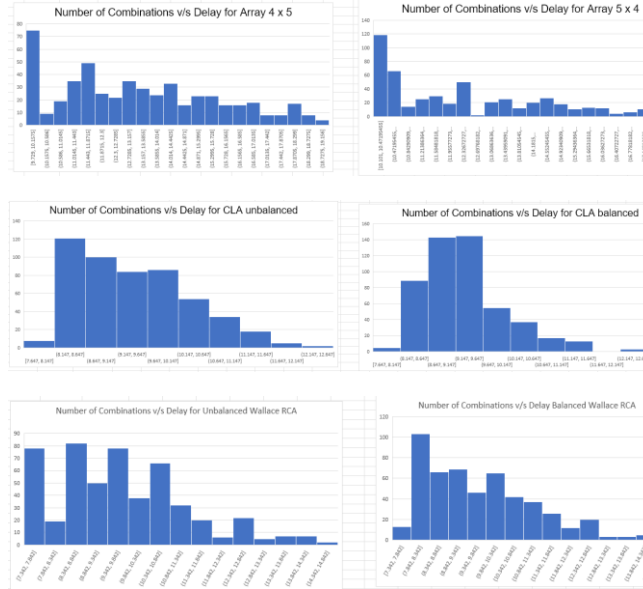
```

$display("\nMax Delay = %0.3f ns (at A=%0d,
B=%0d)", max_delay, worst_A, worst_B);
$finish;
end

endmodule

```

[B] Histograms of delays data for different multipliers



[3] Data-dependent delays of basic gates

Gate	Input Transition	Output Transition	Delay (ns)
NAND2	A=1, B: 0→1	1→0	0.588
	A=1, B: 1→0	0→1	0.915
	B=1, A: 0→1	1→0	0.677
	B=1, A: 1→0	0→1	1.064
NOR2	A=0, B: 0→1	1→0	0.076
	A=0, B: 1→0	0→1	0.275
	B=0, A: 0→1	1→0	0.426
	B=0, A: 1→0	0→1	0.087
XOR2	A=0, B: 0→1	0→1	0.075
	A=0, B: 1→0	1→0	0.855
	A=1, B: 0→1	1→0	0.156
	A=1, B: 1→0	0→1	0.534
	B=0, A: 0→1	0→1	0.086
	B=0, A: 1→0	1→0	0.860
	B=1, A: 0→1	1→0	0.013
	B=1, A: 1→0	0→1	0.381
INV	A: 0→1	1→0	0.372
	A: 1→0	0→1	0.597

REFERENCES

[1] Expression for r_0 taken from Chapter-11, Handbook of Digital CMOS Technology, Circuits, and Systems, Karim Abbas.